

The Plain Text (TXT) File Format: Specifications, AI Applications, and Comparison with JSON and PDF

Introduction

Plain text files (typically with the `.txt` extension) are one of the simplest and most ubiquitous file formats in computing. A text file is essentially a sequence of human-readable characters organized into lines ¹, without any special formatting or binary encoding. This simplicity has made TXT files a fundamental medium for data storage and exchange. In this paper, we provide a comprehensive technical overview of the TXT file format, including its full specifications (character encodings, file structure, line termination conventions, and metadata considerations). We then explore how plain text files are used in artificial intelligence workflows – from natural language processing corpora and data preprocessing to model training, embeddings, and prompt engineering for large language models. In addition, we compare plain text files to two other common formats, JSON and PDF, highlighting the advantages and limitations of each in AI use cases (e.g. human readability, ease of parsing, structural richness, and compression). Throughout, we cite relevant standards (such as Unicode specifications for UTF-8) and historical documentation. Finally, we discuss practical considerations of using text files in domains like robotics or embedded systems (for example, in a robotics project like the Monkey-Head-Project), where resource constraints and simplicity needs influence format choice.

TXT File Format Specifications

Definition and File Structure

A **text file** is defined as a file containing a sequence of characters divided into zero or more *lines*, typically ending in an end-of-line delimiter ² ³. In modern operating systems, the end of the file is determined by the file size recorded in the filesystem, and no explicit end-of-file character is required ⁴. (Historically, in OSes like CP/M and early DOS, a special control character such as `^Z` (0x1A) was used as an end-of-file marker for text files ⁴, but this practice is obsolete in modern systems.) Each line of a text file is typically terminated by a newline character or sequence. The exact line break representation varies by platform: Unix and Unix-like systems (including Linux and modern macOS) use a single line feed character (LF, `0x0A`) as the newline delimiter ⁵, Windows uses a carriage return + line feed pair (CR+LF, `0x0D 0x0A`) ⁶, and classic Mac OS (pre-OSX) used a single carriage return (CR, `0x0D`) to end lines ⁷. Aside from these line break codes, a plain text file contains no other structural markers – it is a flat sequence of characters. By definition, a text file should not contain *NUL* bytes (`0x00`) in the data, as those are not considered printable text characters and often serve as string terminators in programming contexts ⁸. POSIX standards formally define a text file as “a file that contains characters organized into zero or more lines, where each line is a sequence of zero or more non-newline characters plus a terminating newline character” ⁹. In POSIX, no line should exceed a certain length (defined by `LINE_MAX`) and **NUL** characters are not allowed in text files ⁸. This definition underscores that text files are intended for line-oriented readable data.

Text files are stored as a sequence of bytes within a computer file system with minimal structure ³. They have a *type* of “document file” or generic container but do not enforce any formatting beyond the basic line breaks ¹. It is useful to distinguish **plain text content** from the **text file container**: “text file” refers to the file format (or container) that holds plain text, while *plain text* refers to the content itself (the human-readable characters) ¹⁰. At a broad level, computer files are often categorized as either text files or binary files ¹⁰. Text files are readable in text editors or terminals and contain only readable characters (and whitespace/control characters like newline), whereas binary files contain data in machine-readable form not intended for direct human interpretation.

Because text files are so simple, they do not include embedded metadata or complex internal structure. There are no inherent fields for things like title, author, or formatting within a `.txt` file – it’s “just text.” In operating systems like Windows, a file is often identified as a text file by its filename extension (e.g. `.txt` for generic text), though many other extensions (such as `.log` for logs, `.csv` for comma-separated values, or source code extensions like `.py`, `.c`, etc.) also indicate text content ¹¹ ¹². The MIME type associated with plain text is `text/plain` (often with a charset parameter to indicate encoding) ¹³, reinforcing that no specific formatting is assumed beyond plain textual content.

Character Encodings: ASCII, Unicode, and UTF-8

Text files fundamentally store sequences of *characters*, but those characters must be represented as numeric codes (bytes) according to a character encoding scheme. The choice of character encoding is a critical aspect of the TXT format specification.

For much of computing history, the default encoding for text files was ASCII (American Standard Code for Information Interchange). ASCII is a 7-bit encoding standardized in the 1960s that defines 128 characters (0–127), including the Latin alphabet (A–Z, a–z), digits, basic punctuation, space, and a set of control characters (like newline, carriage return, tab, etc.). ASCII covers **American English** text but does not include symbols like the British pound (£), the euro (€), or letters with diacritics used in other languages ¹⁴. In many contexts, ASCII was assumed as the default character set for text files ¹⁴. For example, early internet protocols and programming languages often treated text as ASCII by default. ASCII’s limitation to 128 characters meant that it could not represent characters outside basic English. To support other languages and symbols, various ASCII extensions and other encodings were developed in the late 20th century. These included single-byte encodings like the ISO/IEC 8859 series (ISO-8859-1 through ISO-8859-16) which added support for Western European and other language characters in the 0x80–0xFF byte range ¹⁵, as well as entirely different encodings like **EBCDIC** on IBM mainframes or double-byte encodings (e.g., Shift-JIS for Japanese, GB2312 for Chinese) for languages with larger character sets. Before the advent of Unicode, it was common to choose a text file encoding based on the **locale** or region – for instance, ISO-8859-1 (Latin-1) for Western Europe, or Windows-1252 on English Windows, KOI8-R for Russian, etc. ¹⁵. This led to fragmentation and interoperability problems: a text file saved in one encoding might display garbled characters (“mojibake”) if read under a different encoding expectation.

Unicode emerged as a universal character set standard aimed at including characters from virtually all written languages and symbol systems. The Unicode character repertoire is over a million code points, enabling a single standard to represent text in *Arabic*, *Chinese*, *Devanagari*, and many other scripts, as well as emoji, mathematical symbols, and more ¹⁶. Unicode is not itself an “encoding” in the sense of bytes; rather, it defines code points (unique numeric IDs for each character). To store Unicode text in files, several encoding schemes (Unicode Transformation Formats, UTF) exist. The most common and relevant for plain

text files is **UTF-8** ¹⁷. UTF-8 is a variable-length encoding (ranging from 1 to 4 bytes per character) that has become the de facto standard for text files and content interchange on the web ¹⁸ ¹⁹.

UTF-8 was designed with critical advantages: it is **backwards-compatible with ASCII** and it is self-synchronizing and easily auto-detectable. Backwards compatibility means that the first 128 Unicode code points (U+0000 to U+007F), which correspond exactly to the ASCII characters, are encoded in UTF-8 using a single byte identical to their ASCII value ²⁰. As a consequence, any plain ASCII text file is also a valid UTF-8 file with the same meaning ²¹. This property greatly eased transition – software that only knows about ASCII can often still read UTF-8 text *as long as* the text is only basic English characters. Conversely, software that is Unicode-aware will usually assume UTF-8 by default; many modern tools attempt to read an unknown text file as UTF-8 first, and only if that fails (encounters an invalid byte sequence) do they fall back to legacy encodings ²². UTF-8's self-synchronizing nature (no byte sequence for one character can appear as part of another character) makes it robust for parsing data streams and for handling text with unknown start points ²⁰.

Under UTF-8, characters are encoded in one to four bytes: for instance, all standard ASCII characters (English letters, digits, etc. in U+0000–U+007F) use one byte (their ASCII code) ²³ ²⁴. Characters like Latin letters with accents, Greek, Cyrillic, Hebrew, Arabic (U+0080–U+07FF) use two bytes. Most common Asian scripts and additional symbols (U+0800–U+FFFF) use three bytes, and rare or historic characters, as well as many emoji (U+10000 and above), use four bytes. This scheme can represent the entire Unicode range of 1,112,064 possible code points ²⁵. For example, the letter **A** (U+0041) is 0x41 in ASCII and in UTF-8 (one byte), whereas a character like **漢** (U+6F22, a CJK ideograph) would be 3 bytes in UTF-8 (**E6 BC A2** in hex). The design ensures that no ASCII byte (0x00–0x7F) appears as part of a multi-byte sequence, so ASCII text is unchanged and multi-byte sequences have distinct patterns (bytes with the high bit set) ²⁰. Today, UTF-8 is *dominant* — it is reported that on the internet well over 95% of text content is UTF-8 ¹⁹, and it is the default or required encoding for many standards and protocols. Consequently, modern “text files” generally imply UTF-8 encoding unless stated otherwise.

It is worth noting that other Unicode encodings exist (UTF-16, UTF-32), but these are less commonly used for simple text files. For instance, UTF-16 is used by Windows internally (“Unicode” in Windows text files often means UTF-16 LE with a BOM) ²⁶, and UTF-16 text files will typically begin with a **Byte Order Mark (BOM)** 0xFFFE or 0xFEFF to indicate endianness ²⁷. UTF-8 does not require a BOM (since byte order is not applicable), though some Windows programs prepend a UTF-8 text file with the BOM (0xEF 0xBB 0xBF) as a signature to indicate the file is UTF-8 ²⁸ ²⁹. This practice can help distinguish UTF-8 from legacy 8-bit encodings in some cases, but it is not universally expected and can cause issues with software that does not expect a BOM (for example, a Unix shebang line at the top of a script would be disrupted by a leading BOM) ²⁹. Outside of those cases, plain text files usually rely on external context (conventions, metadata, or user knowledge) to determine the encoding if it's not ASCII. In an “ASCII only” text file, no special tag within the file tells you it's ASCII; it's just assumed or determined by the environment. Similarly, UTF-8 files without BOM are typically recognized by convention or heuristics (UTF-8 is easily auto-detected by trying to interpret bytes; any random 8-bit file is unlikely to be valid UTF-8 if it contains values in certain disallowed ranges or sequences ²²).

In summary, a proper technical specification of a text file must include the **character encoding** being used. Today the most interoperable choice is UTF-8 (which seamlessly handles ASCII content as a subset) ²¹. Older text files might use ASCII or other encodings; these can still be considered “TXT format” but with an important caveat that the *interpretation* of bytes relies on knowing the correct encoding. For completeness,

ASCII itself was formally defined by standards like ANSI X3.4 and has 95 printable characters (letters, digits, punctuation) plus 33 control codes (including newline, carriage return, etc.). Unicode (and UTF-8) incorporate all ASCII characters identically and extend far beyond. We will cite the Unicode Standard (maintained by the Unicode Consortium and ISO/IEC 10646) as the authoritative source for UTF-8 definitions

18 20 .

Line Endings and Newline Conventions

As mentioned, text files are separated into lines by newline delimiters. This is an area of variation across systems that is important in cross-platform contexts. The content of a text file is structured *logically* as lines, but the actual byte values representing “end of line” differ:

- **Unix/Linux and macOS:** newline is represented by **Line Feed** (LF, `\n`, 0x0A). In POSIX-compliant systems, a text file is expected to end with a newline character as well (the last line should terminate with LF) ⁹. If a file is missing a final newline, certain utilities might consider it a minor formatting issue. POSIX defines a line as “a sequence of zero or more non-newline characters plus a terminating newline” ⁹. Thus, an empty line is represented by a lone LF. macOS, since it is Unix-based, follows the same convention. (Notably, **Classic Mac OS** through OS 9 used a different convention: lines terminated with **CR** only ⁷, but this is now historical. Modern macOS switched to LF when it moved to a Unix foundation.)
- **Windows (and DOS before it):** newline is a two-character sequence **CR followed by LF** (Carriage Return 0x0D + Line Feed 0x0A) ⁶. This originates from mechanical teletype systems where CR meant “return carriage to start of line” and LF meant “advance paper by one line.” Windows still inherits this convention. As a result, a text file created on Windows will have `0D 0A` at the end of each line (shown as `\r\n` in many programming languages). When such a file is opened in Unix without conversion, the CR (`\r`) may appear as an extraneous `^M` character at line ends. Conversely, a Unix text file opened in Windows Notepad historically would appear as one long line because Notepad expected the CRLF sequence – this highlights the importance of handling newline conversions correctly in cross-platform scenarios. Many text editors and tools today can handle both conventions, but the difference persists.
- **Old Mac OS:** as noted, lines ended with **CR** (`\r`, 0x0D) only ⁷. This convention is largely obsolete, but you might encounter it in legacy data. Modern text processing tools usually treat an isolated CR or LF as a newline in a tolerant way.
- **Others:** Some older or specialized systems use different approaches. For example, **OpenVMS** and IBM mainframe OSes (z/OS, OS/360) use *record-oriented* text files where each line is a record in the filesystem, possibly prefixed by length fields ³. In those systems, lines are not delimited by special characters at all, but by record boundaries maintained by the system. When such files are transferred to a byte-stream filesystem (like DOS/Windows/Unix), the records are often converted to CRLF or LF-terminated lines. There were also systems where a specific control character or sequence denoted end-of-line (e.g., older printers or terminals might require both CR and LF commands).

Modern text file standards generally revolve around the CR/LF or LF conventions. Many programming languages and libraries provide newline normalization (e.g. opening a file in text mode on Windows will

often translate `\r\n` sequences to `\n` in memory). Tools like Git also handle line ending normalization if configured, to avoid issues when collaborating across OSes.

When looking at a text file in a hex editor, one can immediately see these differences (0A vs 0D0A patterns). It is common that Windows text files contain the two-byte sequence for each line break, whereas on Linux the same file would be one byte shorter per line. Aside from this difference, the content is the same. Notably, many Windows applications tolerate a final line without the CRLF terminator (and some do not automatically add one to the last line) ⁶, so the last line might not end in 0D 0A; this is generally not problematic except for certain strict text processing expectations on Unix.

Metadata and Limitations of Plain Text Files

One of the defining characteristics of plain text files is the **absence of explicit metadata or rich formatting**. A `.txt` file contains only the characters of the text and the necessary control characters (like newline, possibly tab, etc.) to structure it minimally. It does not natively include information like fonts, styling (bold, italics), images, or hyperlinks – features that are common in word processor documents or PDF. It also lacks *embedded metadata* such as author name, creation date, or encoding declaration (the encoding is usually either agreed upon externally or inferred, as discussed above). The only “metadata” needed to interpret a text file is knowledge of its character encoding ³⁰. In most cases, the filesystem and operating system handle ancillary metadata (file modification timestamps, file name, perhaps external descriptors of encoding or language), but the text file content itself is just raw text.

This simplicity is a double-edged sword. On one hand, it means **universality and longevity**: any computer system can store and display a text file, and the content is not tied to a specific software. The lack of complex structure avoids issues like endianness (byte order) or word size differences that can plague binary formats ³¹. Text is also robust against partial data corruption – if a segment of the file is damaged, one can often open the file and still read the remaining text around the corrupted part, because each character is independent (except for multi-byte encoding issues) ³¹. On the other hand, plain text is **limited in expressiveness** and storage efficiency. The term “low entropy” is often applied to text files: they contain a lot of redundant information from a compression standpoint ³². For instance, in English text, certain letters and combinations are very frequent, and ASCII or UTF-8 uses a whole byte for each character regardless. Thus text files “occupy more storage than necessary” for the information they convey ³². A corollary is that text compresses very well – compression algorithms can often reduce a large text file to 20-30% of its original size (or even smaller) by exploiting these patterns. This is why large text corpora are often distributed in compressed form (e.g., `.txt.gz` or `.zip` containing txt), and why text logs can be archived efficiently.

Another consequence of no internal metadata is that *any additional meaning or structure must be encoded in the text itself by convention*. For example, if one needs to represent tabular data in a text file, one might use commas or tabs to separate columns (CSV or TSV format) or fixed column widths. If one needs sections or formatting, one might use human-readable conventions like underlining or markdown syntax. Indeed, many **lightweight markup languages** (Markdown, reStructuredText, HTML in its early usage, etc.) are essentially plain text with certain annotations that can be interpreted by software to add structure or formatting ³³. These remain partly human-readable (you can read a Markdown file and understand most of it even without rendering) and use only plain text characters. Similarly, program source code is kept in text files, relying on programming language syntax (which is plain text) to impart structure. These

approaches leverage the portability of text while layering a structure that machines or humans can interpret by convention.

Plain text files also lack any **built-in encryption or compression** (whereas some other formats optionally include compression of their content streams – PDF, for example, often compresses text in the file). If needed, external tools must be used (e.g., you can compress a text file with gzip, or encrypt it with PGP, but then it's no longer directly a `.txt` that you can open in a text editor without first decompressing/decrypting).

File size and performance: While text files can be extremely large (many gigabytes, e.g. log files or large corpora), accessing and parsing them may be less efficient than binary formats for certain kinds of structured data. For example, searching for a specific record in a massive text file requires scanning sequentially or building an external index, since there's no inherent indexing. Despite this, text is often favored for logs and data dumps because of its openness; one can use common tools (grep, editors, etc.) to search and manipulate it, even if not most efficient.

Filename extension and identification: As noted, `.txt` is the generic extension on Windows recognized as “text file” ¹¹, but Unix does not rely on extensions – it might look at the content (or more typically, simply assume text if the content appears to be mostly printable characters). The magic number (file signature) for a text file is not a fixed byte sequence (unlike many binary formats), but rather by convention, a file is text if it contains no bytes outside the printable character range (with a tolerance for common control chars like newline, tab, form feed). In practice, a heuristics or user knowledge determines if a file is text. Some operating systems or development frameworks may attach additional metadata externally (for example, classic Mac OS used a file type code `TEXT` in the resource fork to mark a file as a text document ⁷).

In summary, the TXT format is defined by its minimalism: sequence of character-encoded bytes with line delimiters, and nothing else. This minimalism yields maximum **human readability** and **cross-platform compatibility**, at the cost of lacking structure and efficiency. Next, we will examine how these properties of text files make them particularly useful (or at times challenging) in the context of artificial intelligence workflows.

Use of TXT Files in AI Workflows

Text files play a foundational role in many AI and machine learning pipelines, especially those involving natural language data. AI workflows often begin with or revolve around raw textual data, and the plain text format is a natural choice for storing and manipulating such data. Below we explore several facets of AI where TXT files are commonly used:

Natural Language Processing (NLP) Corpora

In Natural Language Processing, large bodies of text (corpora) are the fuel for tasks like language modeling, text classification, translation, and more. It is very common for these corpora to be collected or distributed as plain text. For instance, a corpus of news articles or literature might simply be a directory of `.txt` files or one large text file with one sentence or document per line. Researchers and practitioners often start by reading in plain text data and performing tokenization or other preprocessing. A typical scenario is

described in an NLP toolkit: “Your text data usually comes in the form of (long) plain text strings that are stored in one or several files on disk.” ³⁴ . Indeed, many NLP libraries (such as NLTK, spaCy, or custom scripts) provide readers for plaintext corpora – e.g., NLTK’s `PlaintextCorpusReader` which can read a collection of text files and split them into words or sentences.

Consider the **Penn Treebank (PTB)** dataset (a classic corpus of Wall Street Journal text used in language modeling and parsing research). The PTB is distributed as text files (with `.txt` extension); each line in the PTB training file represents one sentence, with tokens separated by spaces ³⁵ . When loading this corpus for a word embedding training example, one reads the text file line by line, splitting on whitespace to get words ³⁵ . Another example: the **IMDB movie review dataset** for sentiment analysis consists of individual text files, each containing one review (often named by an id) ³⁶ . This organization as multiple `.txt` files makes it easy to manipulate reviews individually (for labeling or filtering) using simple file operations.

Large-scale corpora from the web are also handled as text. **Common Crawl**, which collects billions of web pages, publishes extracted plaintext from those pages as WET files (Web Extracted Text) – essentially plain text dumps of the crawled HTML pages ³⁷ . The Common Crawl pipeline moved to this approach around 2013, producing WARC files (raw crawl data), WAT files (metadata), and WET files that contain just the plaintext content of web pages ³⁷ . This plain text is what many NLP projects use to train language models (e.g., GPT-style models) or word embeddings, since the text content is what matters, not the HTML formatting.

When dealing with multilingual text, ensuring the proper encoding (usually UTF-8) is crucial so that characters in languages like Chinese or Arabic are preserved correctly. NLP preprocessing often involves reading text files and normalizing encoding. Tools have to handle cases like byte order marks or differing newline conventions, but these are manageable compared to dealing with binary documents. In fact, many NLP practitioners will convert any proprietary format into plain text as an initial step. For example, if starting with a collection of PDFs or Word documents, one might use an extractor (like `pdftotext` or `doc2txt`) to produce raw `.txt` files for each, because subsequent NLP processing (tokenizing, etc.) is much simpler on plain text ³⁸ . This conversion is a common recommendation: *“If you want to work with formatted, non-plain text sources such as PDFs, Word documents, HTML files, etc., you must convert them to plain text first (e.g., using pdftotext or pandoc).”* ³⁸ .

Plain text corpora are easy to concatenate, sample, or modify with standard tools. One can grep for specific words across them, split them by lines, or use simple scripts to clean the data (remove non-ASCII characters or punctuation) because the data is accessible as text. This flexibility speeds up the iteration in research.

Data Ingestion and Preprocessing

In broader machine learning (not just NLP), text files are often used as an intermediate or initial data source for ingestion. Many datasets are published as text, and many preprocessing pipelines output text. For instance, a common ETL (extract-transform-load) pipeline might scrape data from various sources and dump the results as a newline-separated text file (each line perhaps a JSON string or a raw record) for downstream processing. Even when ultimately a binary format or database is used, the interchange during development is often through text because it’s easier to debug (you can open the file and inspect it).

Data ingestion for AI frequently involves reading features or labels from text-based formats like CSV, JSON, or just plain line-based logs. A simple example: a list of stop words for an NLP task might be in a `.txt` file

(one word per line), which the code reads to filter out those words. Or consider configuration files for data processing – they might be `.txt` or `.ini` files that specify parameters for how to parse data.

During **preprocessing**, textual data might be cleaned (e.g., lowercasing, removing certain lines) using plain text operations. Scripting languages like Python make it easy to read text files line by line for processing. The ubiquitous nature of text means that a wide array of tools (from the Unix command-line utilities to Hadoop's `TextInputFormat`) know how to handle text input.

A concrete AI-related example is preparing a **training corpus**: one might have multiple text sources (say, several ebooks or articles). A preprocessing step could involve concatenating these into a single large text file or normalizing them to a consistent encoding and line format. Another example is **embedding generation pipelines** for search or retrieval: one might have a directory of text documents (e.g., product descriptions or knowledge base articles) which are read in as text and passed into an embedding model to produce vectors for a vector database. In such a pipeline, text files serve as the simple interface between a crawling step and the embedding model input ³⁹. The advantage is that even if upstream or downstream systems change, as long as they produce/consume text, integration is easier than a complex binary protocol.

In robotics and embedded AI contexts, data ingestion might involve reading sensor calibration values or waypoints from a text file on disk, because it's easy to update those without reflashing firmware. For example, an embedded controller might load a `waypoints.txt` containing coordinates for a robot to move to, each coordinate on a line. This is not unlike data ingestion in larger AI systems, just on a smaller scale. The **Monkey-Head-Project**, for instance, being a robotics project, could use a text file to define certain behaviors or calibration parameters for the robotic head. Storing those parameters in a human-readable text file means an engineer can quickly tweak them on the device (or SD card) and reboot, rather than dealing with a binary blob. Text's human readability is a huge plus during iterative development and debugging in AI as well as robotics.

Model Training Datasets (Corpus Handling)

When training models on text data, whether it's a traditional machine learning model or a deep learning model, the training data often originates from text files. For unsupervised language modeling (like training GPT-style models or word embeddings), one typically needs a large text corpus. For example, the training of Word2Vec on the Google News dataset – the data was originally text (news articles). The **One Billion Word Benchmark** for language modeling was released as plain text (shuffled sentences from news, in text files). The **WikiText** datasets (WikiText-103, etc.) are provided as plain text extracted from Wikipedia articles. Even for supervised tasks like classification, the raw text is needed to feed into the model.

Often, after preprocessing, one might convert text to a more efficient binary format for actual training (for speed), but even that conversion (e.g., to `TFRecord` in TensorFlow or to a numpy binary) begins with reading the text. The enduring presence of plain text at the input stage is because labeling and assembling of data are easier in text form. For instance, a developer might maintain a `.txt` file of positive and negative example sentences for a quick experiment in sentiment analysis. It's trivial to edit and version such a file with a diff tool.

When dealing with very large corpora, sometimes text is split into multiple files (e.g., one file per book or per month of data) for manageability. These are still plain text, just chunked. Many frameworks (PyTorch's

`TextDataset`, TensorFlow's text loaders) can stream from text files. As a result, knowledge of text file encoding and newline handling is important to ensure there's no unintended drop of characters (for example, reading in universal newline mode so that both `\n` and `\r\n` are handled).

To give a specific example of corpus handling: the **Penn Treebank** we mentioned, or a dataset like **WikiText-2**, is typically loaded by reading the text file and tokenizing. In a tutorial for training word embeddings, one sees code that opens `ptb.train.txt` and reads all the text into memory, then splits by newline and then by spaces to get tokens ³⁵ ⁴⁰. This demonstrates how straightforward the pipeline is with text – a few standard string operations suffice to prepare the data for model training.

Another angle is that after model training, the *output* might be saved as text. For example, a learned word embedding model (like GloVe) produces vectors for each word. The popular pre-trained GloVe embeddings are distributed as a text file where each line starts with a word followed by the numeric components of its vector ⁴¹. This is convenient because anyone can open that file, inspect a word's vector, or load it with simple code (reading lines and splitting). The line from a tutorial states: "These models are usually distributed as text files. Each line typically contains a word followed by its corresponding vector components (floating-point numbers) separated by spaces." ⁴¹. The fact that even something as numeric as word vectors is shared as a `.txt` underscores the trust in text format for compatibility. (Some embeddings are also offered in binary format for efficiency, but the text version remains common for maximum interoperability.) Similarly, datasets of word relationships or dictionaries for NLP are often plain text (like WordNet's data files, etc.).

Embedding Generation

Embeddings – vector representations of data (text, images, etc.) – are central in modern AI. When generating text embeddings (for words, sentences, or documents), the pipeline invariably starts with text. For example, to compute embeddings for a collection of documents (say, to build a semantic search index), one might have a directory of text files for the documents. A script will iterate over each `.txt` file, read the text, perhaps do some cleaning, and then feed it to an embedding model (like a transformer or a sentence embedding model). The text files here serve as the input API: you could update the content in them or add new files, and the embedding pipeline doesn't need to change to accommodate new data.

A practical instance: suppose you have a knowledge base of FAQs. These might be stored in a text file (or a simple markup). To create embeddings for each FAQ question for use with a vector database, you'd read each question (text) and compute its embedding. The simplicity of text means you can quickly fix typos or update an entry in the source `.txt` and regenerate the embedding. If the data were locked in a PDF, you'd first have to extract text each time.

There's also the case of **embedding model outputs**: As mentioned, pre-trained embeddings (word2vec, GloVe) are often saved as text. Another example is storing *embedding vectors themselves* for debugging – one might output them to a CSV or plain text matrix for inspection. Though typically, vectors are binary or in memory, if you need to manually check a few values, writing them as text (with some precision) is a quick solution.

From a robotics perspective, one might not immediately see "embeddings" since robotics deals with sensor data, but consider an embedded AI that does language understanding (e.g., voice commands processed on-robot). It might use text embeddings of recognized commands to compare to known intents. Those

known intents could be stored as plain text phrases in a file, loaded at startup, embedded, and then the robot compares incoming command embeddings to the stored ones. The advantage: updating the list of commands is as simple as editing a text file of commands, rather than recompiling code. This is speculative, but it illustrates how text can be the bridge between AI models and configurable data in a system like a robot.

Prompt Engineering and LLM Fine-Tuning

The recent rise of large language models (LLMs) like GPT-3, GPT-4 and others has given prominence to **prompt engineering** – the craft of designing input prompts to elicit desired responses. Prompts themselves are chunks of text. Developers and researchers often maintain collections of prompt examples in text form. For instance, you might have a file `prompts.txt` where you iterate on various phrasings to see which works better. The prompt templates used in a system (like “You are an assistant that...”) are usually stored as strings in configuration or files, which is essentially plain text.

When systematically evaluating prompts, one might create a JSON or CSV with prompts and expected outputs. However, for quick experiments or manual prompt building, nothing beats a simple text editor and `.txt` file. Teams also share prompts via text (Slack messages, plain text in papers, etc.), underlining that the medium for exchange is text.

In LLM **fine-tuning** (where an LLM is further trained on a custom dataset, such as instruction-response pairs), the training data typically consists of textual examples of prompts and completions. OpenAI’s fine-tuning API, for example, expects the training data to be in a JSON Lines file (`.jsonl`) where each line is a JSON object with a `"prompt"` field and a `"completion"` field containing text⁴². JSONL is still a text-based format – effectively one JSON per line. The reason JSON is used here (instead of raw text) is to clearly delimit the parts of each training example and possibly include metadata. However, one could also fine-tune by constructing a single giant text where prompts and responses are delimited by special tokens (this was done in some early instruction-tuning research). In either case, the model ultimately sees plain text (with special tokens if any) as input. The advantage of JSONL is that it’s machine-parseable and easy for the fine-tune system to ensure prompt and completion are separated correctly⁴². But it’s still human-readable text – one can open the `.jsonl` and inspect examples easily. This highlights a theme: even when more structure is needed, we often choose a text-based structured format (like JSON or CSV) instead of a binary format, to preserve human readability and ease of use during development.

Prompt engineering often involves iterative refinement, and people might keep notes or documentation in Markdown (which is again plain text with formatting syntax). The fact that prompts are language data means they are inherently text; it wouldn’t make sense to use a binary format for them since you’d lose the direct insight (and you can’t “read” a prompt if it’s not text). Some advanced prompt-engineering setups use templates and placeholders (like `{variable}` in a prompt string) which could be stored in a text template file.

Finally, consider logging the interactions with an LLM during reinforcement learning from human feedback (RLHF) or other fine-tuning processes – logs often go to text files. If you run a fine-tuning epoch and want to see how the model’s outputs compare to expected completions, you might save a text log of prompt, model output, expected output for analysis.

In all these AI workflow stages – from raw data corpus to engineered prompts – text files are ubiquitous because they align with the nature of the data (language) and offer the path of least resistance for manipulation. However, text is not the only format in use. JSON and PDF, among others, also appear in AI pipelines, serving different roles. We will now compare TXT with JSON and PDF formats to clarify when each is advantageous or limiting in an AI context.

Comparing TXT, JSON, and PDF in AI Use Cases

Plain text (`.txt`), JSON, and PDF represent three very different kinds of file formats, each with distinct features. Here we analyze them across several dimensions relevant to AI workflows: human readability, ease of parsing (for machines), structural richness, compression/efficiency, and suitability for various tasks. Table 1 summarizes key differences, followed by elaboration below:

Table 1. Comparison of plain text (TXT), JSON, and PDF formats in the context of AI data.

Format	Human Readability	Machine Parsing	Structure & Metadata	Storage Efficiency	Typical AI Use
Plain Text (TXT)	High – directly readable plain language (if content is natural language). Easiest for humans to skim or edit. ¹	Easy to read sequentially, but <i>unstructured</i> – custom parsing needed for anything beyond raw text. No inherent indexing or data types ⁴³ ⁴⁴ .	Minimal structure (just lines). No embedded metadata or formatting. Can impose structure via conventions (e.g. CSV, markup) but not enforced by format.	Low entropy (redundant), thus larger file sizes, but compresses extremely well ³² . No built-in compression.	Storing corpora, logs, simple config. Input for NLP models, corpora for training, intermediate data exchange. Great for quick prototyping and debugging in AI.

Format	Human Readability	Machine Parsing	Structure & Metadata	Storage Efficiency	Typical AI Use
JSON (JavaScript Object Notation)	High – still plain text (Unicode), with readable key-value syntax. Somewhat verbose but human-understandable (especially to programmers) ⁴⁵ .	Very easy with standard libraries – parse into native data structures with one call. Self-describing structure (keys, arrays) means no custom parsing rules needed ⁴⁴ .	Structured (hierarchical). Supports nested objects, arrays, strings, numbers, booleans. No display formatting, but can include metadata fields as keys. Rigid syntax (commas, braces) must be correct. No binary data (unless encoded as text).	More verbose than binary due to repeated keys and quotes. Still reasonably compact for moderate data. Compresses well (since text). No native compression, but often used in compressed archives if large.	Structured data exchange (e.g. model hyperparameters, annotated datasets). Common for AI dataset distribution (JSONL for prompts/labels). Excellent for when data has structure (keyed fields) in addition to text content.

Format	Human Readability	Machine Parsing	Structure & Metadata	Storage Efficiency	Typical AI Use
PDF (Portable Document Format)	Medium to Low – intended for humans to read on-screen or paper, but the file itself is not plain text. Requires a viewer; raw PDF content is not meant to be directly human-editable.	Difficult – complex format. Parsing PDF for text requires dedicated libraries. PDF internal structure includes objects, streams (often compressed), cross-references – not trivial to extract or manipulate text programmatically ⁴⁷ ⁴⁸ .	Page-oriented formatted document. Can embed fonts, images, graphics. Has metadata (title, author, etc.). Fixed layout – great for presentation, not for data interchange. Text is stored in content streams and not easily accessible as a continuous sequence (may be split into pieces by layout).	Can be efficient for mixed media documents (images, compressed, repeated elements etc.), but for plain text it has significant overhead (font embedding, structure). Often already compressed internally. Not easily compressible further if streams are compressed.	Used to present AI results (reports, papers) or as input for document analysis tasks. Not used to train language models directly – text must be extracted first ³⁸ . Useful when layout matters (e.g., vision+text tasks), but not as a native training data format for NLP.

Human Readability: Plain text is the baseline for human readability – if the content is natural language, a `.txt` file can be opened in any editor and immediately understood (assuming language/encoding knowledge). JSON, while it contains extra symbols `{}` and `[ : ]`, is still designed to be human-readable text; one can usually decipher the structure and values, especially if pretty-printed. JSON's human readability is a reason for its popularity over binary alternatives like Protocol Buffers in many scenarios. PDF, however, is not designed to be opened in a text editor for reading – if you do, you will see a mix of readable tokens (PDF keywords and perhaps some text strings if not compressed) and lots of unreadable bytes (due to compression or binary data for images, etc.). The content that a human cares about in a PDF (the rendered text and images) requires a PDF viewer or extraction tool to see. So, in terms of directly opening the file and reading, PDF is effectively not human-readable. It's optimized for viewing in its final form (layout on a page), which is a different notion of readability (visual readability vs. source readability). For AI practitioners, being able to quickly read the raw data is often important during development; here TXT (and JSON) have the clear advantage.

Ease of Machine Parsing: This is a crucial point for AI workflows where you need to load data into programs. Plain text with *no structure* can actually be the hardest to parse if the data has multiple fields or

records – you have to write custom code or use regex to split the content because the file doesn't tell you how data is separated. As one Stack Overflow answer succinctly put it, if you send data as plain text, “*you would have to make up your own system for separating different variables... and handle exceptions when the text itself contains the delimiter*” ⁴³. In other words, flat text requires an agreed convention for parsing (like CSV uses commas, etc.), whereas JSON and similar formats have a *standardized* parsing logic. JSON can be parsed by essentially all programming languages using a library: it directly gives you a dictionary/array structure in memory ⁴⁶. The same goes for formats like XML (though XML is more verbose). JSON and XML were explicitly created to make data exchange machine-friendly while still being human-readable. They include things like quoted strings and escaping to ensure that the structure is unambiguous (no need to worry that a `}` is actually data – it's always a structure delimiter in JSON). The advantage in AI context is huge: if your dataset is in JSON, you can load it and immediately access fields like `text`, `label`, etc., with well-defined types. If the same data were in a `.txt` with lines like `"positive\tI loved this movie"`, you'd need to parse the tab and know that the first token is label and second is text. That works but is ad hoc.

Stack Overflow answers emphasize that *structured formats (JSON/XML)* allow direct mapping to native data structures and come with indexing and query advantages ⁴⁶ ⁴⁴. For example, in JSON you might have an array of objects, and you can loop through them or access by key. In a plain text scenario, you might have to know “line 10 corresponds to item 10” and if you need item 500 you have to maybe read all previous lines or use an index file. JSON can also handle nested data which plain text cannot represent without a custom syntax.

PDF is on the opposite extreme: parsing a PDF to get the text out is non-trivial. PDF has an internal structure (it's actually a sort of marked-up text for its objects), but it's a *binary* in effect, often with compression, and requires understanding of the PDF specification (ISO 32000). To get text from a PDF, an AI developer might use a library (like PyPDF2, PDFPlumber, or Adobe's API) which itself has to interpret the layout, decode text streams, apply fonts to get Unicode characters, etc. It's a heavy process. If the PDF contains scanned pages, it's just images and you'd need OCR to get text – even more complexity. So from a parsing standpoint, PDF is the least accessible format for data ingestion. It's meant to be final output, not intermediate data. There are entire papers and tools about the pitfalls of PDF parsing ⁴⁹ ⁵⁰ because many things can go wrong (missing fonts, weird encodings, etc.). In contrast, JSON and TXT won't have those issues (the worst might be an encoding issue, but nothing as complex as PDF).

Structure & Metadata: Plain text has virtually no structure beyond sequential characters and line breaks. There's no concept of “this part is a title” or “this part is a field name” unless the text itself encodes that (like a line starting with “Title:” is a convention but not enforced by format). No intrinsic metadata means a `.txt` file doesn't carry info about its source or author, aside from what might be written in the text (some people put a header in comment form inside text files, but that's just content as well). JSON, on the other hand, is all about structure: it represents data objects. It can naturally include what we might call metadata as fields. For example, in an AI dataset JSON, you might have: `{"text": "hello world", "language": "en", "source": "tweets"}` – here language and source are metadata about the text, encoded as part of the JSON data. JSON does not allow specifying display properties (no fonts or styling) – it's purely logical structure. This makes JSON excellent for **structured datasets** (like annotations, lists of examples with labels) because each piece of data can carry its context.

PDF is rich in a different way: it has a structure for layout (pages, paragraphs, etc.) and also a metadata section (PDF has an Info dictionary where author, creation date, etc., can be stored). However, that structure is not meant to convey logical relationships like “this paragraph is a separate training example from the next” – it’s more about visual placement. PDF can be tagged (for accessibility) to indicate which text is a heading or a table, but many PDFs lack proper tagging. The result is that logically, for AI purposes, PDF structure is usually *inconvenient*. A model cares about the sequence of text, but a PDF might have that sequence split into two columns or with a figure in between in reading order. The structure is aimed at faithfully reproducing a formatted document. For instance, if an AI is reading research papers (PDFs), one major task is extracting the plain text from them, discarding the layout. So ironically, to use PDFs in NLP, we strip them back to plain text as much as possible.

One key difference: **extensibility**. JSON can be extended (you can add more keys without breaking readers that ignore unknown keys, typically). Text can’t be “extended” except by changing its content and agreeing on new conventions (which might break older parsers). PDF has a versioned spec that allows new features, but again, that’s not usually relevant to AI data content, more to display features.

Storage and Compression: Plain text is not space-efficient for large data. For example, representing a large numeric array in text (like a list of embedding vectors) will consume a lot more bytes than a binary matrix. JSON is also inefficient in that sense (numbers as ASCII digits, repeated keys, etc.), and PDFs can be inefficient if not compressed (embedding full fonts, etc.). However, both TXT and JSON benefit from being highly compressible. A large text file can often be compressed 5-10x smaller with gzip/ZIP because of repeated patterns (space, common words, etc.) ³². JSON similarly compresses well; for instance, repeating key names in a JSON make it quite compressible (there are also specialized compressors that even do JSON-aware compression).

PDF often already uses compression internally (most PDFs apply Flate/ZIP compression to streams of text and images). Thus, a PDF file might not shrink much more if you zip it because it’s already compressed. In scenarios where data size matters (like transmitting a dataset), one might consider binary formats. For textual data, an interesting middle ground is to compress the text files and still use them. This is done in some ML workflows where you keep data in compressed text format and use streaming decompression.

In-memory efficiency is also a factor: when you parse text or JSON, you get perhaps larger memory usage (strings etc.) compared to a binary format that maps directly to memory structures. For extremely large datasets, formats like TFRecord (binary) or Parquet (columnar binary) are used for performance – but those sacrifice human readability for efficiency.

Suitability for AI tasks: - **TXT** is ideal when the primary content is natural language or when simplicity and human-inspection are priorities. For example, training a language model on novel text might start with a bunch of `.txt` files of novels. Logging model predictions (text outputs) is usually done to a text file. Configuring experiments via text config files (or simple `.txt` lists of parameters) is common. One drawback of raw text in AI is when you need to pair inputs with labels or metadata – then you either maintain parallel text files (e.g., one file with inputs, one with labels in same order) or combine them in text in a structured way, which can become error-prone. That’s where JSON or CSV often take over. - **JSON** shines for structured AI datasets. Many popular datasets come as JSON: e.g., Microsoft COCO’s annotations are in JSON (images have associated captions or labels in a JSON file), many Hugging Face datasets use JSON or JSON Lines. JSON’s advantage is that it can store text along with numbers and categorical data all in one file, preserving the association. In prompt engineering or model fine-tuning, JSONL (each line a JSON) is used exactly to

store prompt-completion pairs clearly ⁴². It's also used for things like storing model predictions with associated IDs. JSON is generally not used for the raw text corpora (because it would be a lot of overhead to wrap every sentence in quotes and braces), but for labeled or structured corpora, yes. One specific example: OpenAI's fine-tune format expects a JSONL with `{"prompt": "...\\n\\n###\\n\\n", "completion": "..."}` – this encapsulation ensures that even if prompts or completions contain newlines or special characters, they are safely represented as JSON strings.

JSON's limitations include that it's not great for very large data (parsing a 10GB JSON file might be slow or memory heavy – sometimes line-delimited JSON is used to allow streaming). But it's language-independent and widely supported ⁵¹, which is beneficial when AI researchers use different tools (Python, Java, etc. can all read JSON easily).

- **PDF** is rarely, if ever, used *directly* to feed an NLP model. However, in certain AI use cases like **document intelligence**, computer vision, or multi-modal models, PDFs are inputs. For example, an AI system that answers questions about documents might take PDFs, run an OCR or PDF-text extractor to get text, possibly analyze layout (tables, headings). So PDF is used in AI mostly as a source format to be analyzed (in tasks like document classification, PDF parsing, etc.), not as a training data store for language itself. There are exceptions: if one were training a model to *learn to render* text (like learning a PDF generator, or to do document layout analysis, etc.), one might feed PDFs in some form to the model. But typically one would convert the PDF to images or text for the model anyway. For robotics or embedded systems, PDF has virtually no role – one wouldn't have a microcontroller read a PDF manual for instructions (one would extract the needed data into a simpler form beforehand). PDF is heavy, requiring megabytes of library code to parse, which is unsuitable for embedded. In robotics, if anything, PDFs might be used as offline references for humans (like a robot's user manual), not as part of the robot's data processing.

Advantages summary: Plain text is unsurpassed for human accessibility and quick use. JSON provides a great blend of structure and readability, making it easier to write correct parsers and avoid mistakes in data alignment ⁴⁶ ⁴⁴. PDF's advantage is in preserving complex information (like images, exact layout, fonts) – something neither plain text nor JSON can do. For instance, if an AI approach requires the original formatting (like reading a math equation as rendered), only PDF (or image formats) capture that. But that advantage is situational and comes with the cost of complexity.

Limitations summary: TXT's lack of structure can become a limitation as a project grows – e.g., maintaining a large training dataset in a single text file with markers can be cumbersome (you're effectively inventing a mini-format). JSON's verbosity and text nature mean it's not the fastest or most compact for extremely large-scale or production deployment (there, binary serialization might replace it). PDFs, while compact for distribution, are essentially "black boxes" to most AI algorithms until processed – a limitation in any pipeline that tries to use them.

To ground this in a concrete example, imagine an AI project in an embedded robotics context (like the Monkey-Head-Project): Suppose the project includes a small neural network that runs on the robot for detecting certain phrases or commands. The developers might have a list of command phrases and their meanings. In plain text, they could store this as a `commands.txt` with lines like `turn left = ROTATE: -90` (just as an illustrative format). The robot's code would parse each line at startup. This is simple, and a human can edit `commands.txt` on the fly. If they used JSON, they might have a `commands.json` like `[ {"phrase": "turn left", "action": "rotate", "angle":`

`-90}, ... ]` which is more structured and self-explanatory to parse (and could be extended to include synonyms or languages easily by adding keys). JSON would make the parsing code a bit easier (just use a JSON library and loop through objects), at the expense of being slightly heavier for the microcontroller to handle if code space is limited. A PDF here would make no sense – it would be absurd to have the robot load a PDF to get its command list. That highlights how PDF is geared toward final human consumption (maybe the command list PDF is given to users to read, but not to the robot). Meanwhile, the advantage of TXT in this embedded scenario is minimal overhead; JSON's advantage is clarity and structure which reduce bugs (e.g., no need to worry about someone writing a phrase that contains '=' and breaking the custom parse – JSON would handle that by putting the phrase in quotes properly).

Security Considerations

One tangential point: in some applications, using text vs binary can have security implications (like JSON and text are susceptible to injection attacks if not handled carefully – e.g., feeding user input into a text command could break format). PDF can contain active content (JavaScript, links) that pose security risks if automatically parsed. In AI data, this is usually not a big concern unless dealing with user-provided PDFs or maliciously crafted JSON. But it's worth noting that plain text is pretty safe (worst might be escape sequences in a terminal), whereas parsing a PDF from an untrusted source must be done with robust libraries due to its complexity.

Use in Robotics and Embedded Systems: The Monkey-Head-Project Example

Robotics and embedded systems often have constraints (limited memory, slower CPUs, real-time requirements) that influence the choice of data formats. In such environments, simplicity is key – which often means plain text or other lightweight formats are preferred for configuration and logging. The **Monkey-Head-Project**, a hypothetical (or internal) robotics project, can serve as an illustrative case. In this project, suppose we have an embedded controller managing a robot head with certain behaviors. We might use text files in several ways:

- **Configuration Files:** A file like `mhp_config.txt` might contain settings (e.g., servo calibration values, operation modes, network addresses). As a plain text file, it's easy for an engineer to edit on the device (no specialized tool needed) and easy for the firmware to parse using simple string routines. If the project grows, they might switch to a slightly more structured text format like JSON or YAML (YAML is another text-based format often used in embedded configs). But even JSON on a microcontroller is feasible if a JSON parser is available – it provides clearer structure at some cost. For instance, if configuration is hierarchical (say, multiple profiles of settings), JSON might be cleaner. However, TXT will be more lightweight in code (no JSON library required, just basic parsing). The trade-off is between code complexity and data clarity.
- **Behavior Scripts:** Perhaps the Monkey-Head-Project allows defining sequences of movements or dialogues via script files. Using a `.txt` script (maybe a simple custom scripting language or just a list of commands like "NOD 3 times" or "SAY Hello") could allow quick experimentation without reflashing code. This is analogous to prompt engineering but for robotic actions – you tweak a text file to change behavior.

- **Logging:** Robots often log sensor readings, errors, or debug info. Logging to a text file (`log.txt`) is straightforward; the logs can later be retrieved and inspected to diagnose issues. In an AI context, if the robot has an AI module (like speech recognition), it might log recognized words as text. Text logs are extremely valuable in embedded development because they provide human-readable traces of what happened. JSON logging is also possible (structured logs), but on embedded systems it might be too verbose or heavy; plain text is usually fine. The downside is that parsing text logs programmatically after the fact (for analytics) might be harder than if they were JSON or CSV. But given resource constraints, many embedded projects accept that trade-off.
- **Inter-device Communication:** Sometimes embedded systems use text-based protocols (like sending commands over serial as text lines) because they are easy to implement and debug (you can literally watch the commands in a serial console). For example, if the Monkey-Head-Project's head has to communicate with a base station, they might design a simple text protocol: e.g., the head sends `"STATUS OK\n"` periodically or receives commands like `"MOVE 30\n"`. These are essentially text messages. Alternatives could be binary protocols (more efficient but require careful packing/unpacking) or JSON messages (self-descriptive but larger). In robotics, there's a mix: some systems use very compact binary protocols for efficiency (like CAN bus messages), others (especially DIY or less performance-critical) use text protocols for ease of development. A text protocol can be debugged with a common terminal tool, which is very handy.

In all these cases, the plain text approach leverages the fact that humans are often in the loop in robotics development – being able to open a file or watch a stream and immediately understand it speeds up development. Similarly, AI researchers often favor text-based data during development for the same reason, even if they might switch to binary for production if needed.

One must also consider the **limitations** in embedded use: text parsing can be CPU-intensive if the text is large and complex. If Monkey-Head-Project had to parse a 10,000-line text file on a tiny microcontroller at boot, that could introduce latency. A binary format (say a pre-parsed config in flash) might load faster. But for small to moderate config sizes, this is usually fine. Also, storing large text files uses more flash memory than a binary equivalent. For example, storing a dictionary of 1000 words might take more space in a text list vs a compact binary table. Again, these are trade-offs between human-friendly vs machine-efficient.

For an AI on the robot (if any), say the robot has a small ML model for gesture recognition – that model's weights would be in binary format (something like a `.bin` or in code) because that's numeric data not suitable for text. But any labels or category names (like if output classes have names) might be stored in a text file. For example, if a vision model on the robot classifies objects, a `labels.txt` could map label indices to human-readable names. This is common even in large systems (e.g., ImageNet class names in a text file).

In the context of AI use in robotics, one can consider how each format might come into play: - **TXT:** for all reasons above, plus ease of updating the AI's knowledge base in the field (imagine updating a list of stop words or responses through a text file update). - **JSON:** possibly for more complex structured settings or if the robot had to share data with a cloud service in a structured way (JSON is often used in IoT for cloud messages). - **PDF:** probably only if the robot needs to show a user a document or if the design specs or test results are stored as PDF for engineers. Not directly used by the robot's code.

To draw a parallel to how AI uses these formats in general: - JSON vs TXT: There's a known preference to use JSON (or XML) when data complexity grows because *"they can be interpreted directly into native data structures... all data has indexes... If using plain text, you'd have to devise your own parsing which gets impractical as data complexity grows."* ⁵² ⁵³ . That quote from Stack Overflow summarizes why JSON is chosen over ad-hoc text in many cases. In robotics, we see the same pattern: if the config or data remains simple and fixed, a custom text is fine; if it grows or needs to ensure no parsing ambiguity, JSON might be introduced.

- PDF vs others: The presence of PDF in AI pipelines is usually at the fringes (input documents or output reports). For example, after developing an AI model, one might publish a PDF report or paper describing it, embedding results and charts. The model itself, however, was trained on text or JSON data. Or, if doing an AI that reads invoices, you have PDF invoices as inputs, but then you output the extracted fields as JSON or CSV for downstream use.

In summary, each format has a niche: - **TXT**: best for raw text data and scenarios requiring simplicity and human access. In AI: source text for NLP, simple logs, quick experiments. - **JSON**: best when structured data needs to accompany text or when interoperability is needed. In AI: annotated datasets, config files, web data exchange (APIs often use JSON, so if your AI service calls an API or serves one, JSON is standard). - **PDF**: best for finalized human-facing documents or tasks needing exact fidelity to original formatting (in AI: OCR tasks, document Q&A, etc., or publishing results).

By understanding these differences, practitioners can choose the appropriate format for each stage of an AI project. Often, an AI project will involve *multiple* of these: e.g., gather raw data as PDFs, extract to TXT for training, store annotations in JSON, log results in TXT, and publish a PDF report – all in one workflow. Each format complements the others when used for what it's best at.

Conclusion

Plain text files have endured through decades of technological change due to their simplicity, transparency, and broad compatibility. We have detailed the technical specifications of TXT files – from character encodings (ASCII's legacy and UTF-8's dominance) to line ending conventions and the near absence of intrinsic metadata – and we have seen how these traits make text files both powerful and limited. In artificial intelligence workflows, these "limitations" of plain text (lack of structure, higher storage cost) often become strengths in the early stages of development: a text corpus is easy to obtain and feed into an NLP model; a `.txt` log or dataset is trivial to inspect and debug; a prompt for an LLM is written in plain text because that is exactly the medium the model understands.

At the same time, we recognized that as soon as additional complexity enters (like labeling, or multi-field records), formats like JSON offer a structured, self-describing approach that reduces parsing effort and errors. JSON's human-readable key-value structure strikes a balance that has made it a staple for dataset interchange in AI, especially for structured annotations and configurations ⁵² ⁴⁴ . We contrasted this with PDF, which serves a very different purpose: preserving a document's appearance. PDF is invaluable when visual fidelity or distribution is the goal (e.g., sharing a research paper, scanning printed forms for an AI to read), but it is ill-suited as an intermediate data format in AI workflows. The need to convert PDFs to text before analysis in NLP is a testament to the primacy of plain text for language tasks ³⁸ .

In domains like robotics and embedded systems, we highlighted that plain text remains a go-to format for many of the same reasons – ease of use and clarity. However, developers must judiciously choose when the

overhead of parsing text is acceptable and when a more structured or binary approach is warranted. The Monkey-Head-Project example illustrated how a simple text interface for commands or config can expedite development and troubleshooting. As AI techniques increasingly integrate into embedded and robotic systems (for instance, a robot with natural language interaction or on-board machine learning), the use of text files for bridging human intentions and machine processing will likely continue. Even constrained devices benefit from the fact that a text file “needs no additional metadata (other than knowledge of its character set) to assist the reader in interpretation” ³⁰ – a principle that applies equally if the “reader” is a human engineer or a piece of code on a robot.

In summary, plain text, JSON, and PDF each play distinct and complementary roles in the storage and handling of information in AI. Plain text is unrivaled for its **simplicity and directness** in representing language data; JSON provides **structured clarity** for complex data exchanges; PDF ensures **fidelity and human-ready presentation** when needed. Understanding their specifications and trade-offs allows AI practitioners to use each format to its strength – whether it’s feeding millions of UTF-8 encoded sentences into a language model, curating a JSONL file of prompt-response examples for fine-tuning, or extracting intelligence from a stack of PDF documents. Despite the rapid evolution in AI, the humble text file remains deeply woven into the fabric of data science and engineering, proving that in many cases, the simplest format is sufficient, and indeed optimal, for the task at hand.

References:

- Unicode Consortium – *The Unicode Standard* (definition of UTF-8 and character encoding) ¹⁸ ²⁰
- IEEE POSIX Standard 1003.1 (2017) – definition of text file (no NULs, newline-terminated lines) ⁸
- Wikipedia – *Text file* (general overview of text files, encodings, newline conventions) ¹⁴ ⁶
- Wikipedia – *UTF-8* (character encoding details and ASCII compatibility) ²⁰
- Stack Overflow – *Why JSON and not TXT* (discussion of parsing advantages of JSON vs plain text) ⁴⁶
⁴³
- tmToolkit Documentation – *Working with text corpora* (on typical plaintext data sources in NLP) ³⁴
³⁸
- Dive into Deep Learning – *Word Embedding Dataset* (example of text file format for NLP corpus) ³⁵
- Reddit (r/programming) – *PDF is a hybrid format (text and binary)* (comments on PDF internal structure and parsing difficulty) ⁴⁷ ⁴⁸
- OpenAI Fine-Tuning Guide – (JSONL format requirement for training data) ⁴²
- Appl. Machine Learning Blog – *Pre-trained Embeddings Distribution* (embedding files often in text format) ⁴¹

1 2 3 4 5 6 7 9 10 11 12 13 14 15 16 17 21 22 26 27 28 29 30 31 32 33 Text file -

Wikipedia

https://en.wikipedia.org/wiki/Text_file

8 Definitions

http://pubs.opengroup.org/onlinepubs/9699919799/basedefs/V1_chap03.html

18 19 20 25 UTF-8 - Wikipedia

<https://en.wikipedia.org/wiki/UTF-8>

23 24 Mastering the UTF-8 Character Set in Programming

<https://www.dhiwise.com/blog/design-converter/a-developers-guide-to-the-utf-8-character-set>

34 38 Working with text corpora — tmtoolkit documentation

https://tmtoolkit.readthedocs.io/en/latest/text_corpora.html

35 40 15.3. The Dataset for Pretraining Word Embeddings — Dive into Deep Learning 1.0.3 documentation

https://d2l.ai/chapter_natural-language-processing-pretraining/word-embedding-dataset.html

36 Text Data Processing with Deep Learning (Word Embedding,RNN ...

<https://www.linkedin.com/pulse/text-data-processing-deep-learning-word-embeddingrnn- lstm-rahul-jain>

37 Text Corpus for NLP

<https://devopedia.org/text-corpus-for-nlp>

39 How to deploy NLP: Text embeddings and vector search - Elastic

<https://www.elastic.co/search-labs/blog/how-to-deploy-nlp-text-embeddings-and-vector-search>

41 Using Pre-trained Word Embedding Models

<https://apxml.com/courses/nlp-fundamentals/chapter-4-nlp-word-embeddings/using-pretrained-embeddings>

42 OpenAI's fine-tuning feature - Medium

<https://medium.com/@jimwang3589/openais-fine-tuning-feature-2f61ef5d3eba>

43 44 46 52 53 xml - Why json and not txt - Stack Overflow

<https://stackoverflow.com/questions/44509942/why-json-and-not-txt>

45 51 JSON - Wikipedia

<https://en.wikipedia.org/wiki/JSON>

47 48 Basic Structure of PDF Format : r/programming

https://www.reddit.com/r/programming/comments/p30d55/basic_structure_of_pdf_format/

49 [PDF] Grammar-Based Specification and Parsing of Binary File Formats

<https://ijdc.net/index.php/ijdc/article/download/217/276/911>

50 Pitfalls of PDF parsing and guidelines for file formats | Blog

<https://gendignoux.com/blog/2016/10/19/pdf-parsing-pitfalls.html>